

LOGIC.md — Полная логика приложения EVA

Этот документ — единственный источник истины по логике работы приложения. Все тексты сообщений и кнопок вынесены в отдельный файл `TEXTS.md` — чтобы их можно было править отдельно от логики.

0. Стек и общая архитектура

- Frontend:** Next.js 14 (App Router), React 18, TypeScript, Tailwind CSS, Framer Motion.
- WebApp UI:** Telegram Mini App (TMA), запускается через `@twa-dev/sdk`.
- Backend API:** Next.js Route Handlers (`app/api/...`).
- БД:** Supabase (Postgres) + Edge Functions на Deno.
- Бот:** Telegram Bot API через webhook (`/api/webhook/telegram`).
- Отложенные задачи:** таблица `bot_tasks_queue` + Edge Function `process-bot-queue`, запускаемая каждую минуту через `pg_cron` (или внешний планировщик).

Ключевые URL-константы (из `.env`)

Переменная	Значение	Где используется
<code>TELEGRAM_BOT_TOKEN</code>	токен бота	сервер: запросы к Telegram API
<code>TELEGRAM_CHANNEL_ID</code>	id канала (<code>-100...</code>)	проверка подписки <code>getChatMember</code>
<code>TELEGRAM_CHANNEL_URL</code>	<code>https://t.me/sprosievu</code>	ссылка для кнопки «Подписаться»
<code>NEXT_PUBLIC_BOT_USERNAME</code>	<code>sprosievubot</code>	реферальные ссылки <code>t.me/<username>?start=ref_<tgId></code>
<code>NEXT_PUBLIC_APP_URL</code> / <code>VERCEL_URL</code>	домен webapp	для <code>web_app.url</code> в кнопках бота и в URL картинок
<code>SUPABASE_URL</code> , <code>SUPABASE_SERVICE_ROLE_KEY</code> , <code>SUPABASE_JWT_SECRET</code>	доступ к БД и подпись JWT	API + Edge Functions

Переменная	Значение	Где используется
ADMIN_PIN	0999	вход в админку (раньше было 2026 — заменить!)
GIFT_VIDEO_FILE_ID	будет получен позже	id видео-подарка из Telegram

1. Структура базы данных

Это **актуальная схема** на основе скриншотов Supabase, которые ты прислал. Старые миграции (`001_init_schema.sql`) уже не отражают реальность — изменены имена колонок.

Таблица `profiles`

Один пользователь = одна запись. Создаётся при первом входе в бот (`/start`).

Колонка	Тип	Назначение (комментарий)
<code>id</code>	UUID, PK	Внутренний идентификатор. На него ссылается всё остальное.
<code>tg_id</code>	bigint, UNIQUE	Telegram ID пользователя. По нему ищем в API и боте.
<code>username</code>	text	Telegram username (без <code>@</code>). Может быть NULL.
<code>first_name</code>	text	Имя из Telegram. Используется в обращении в боте («Рада видеть тебя снова, [Имя]!»).
<code>last_name</code>	text	Фамилия из Telegram (необязательно).
<code>avatar_url</code>	text	URL фото профиля из Telegram.
<code>is_subscribed</code>	bool	Флаг подписки на канал. Перепроверяется каждый раз при открытии webapp (см. п. 2). Здесь только кешированное значение.
<code>current_step</code>	int4	На каком вопросе теста остановился пользователь (0–24). NULL — тест ещё не начат или уже завершён.
<code>question_order</code>	int2[]	Случайный порядок вопросов для конкретного пользователя. Генерируется один раз перед первым вопросом, чтобы при возврате на тест порядок не менялся. NULL — тест ещё не открывался.
<code>last_test_date</code>	timestampz	Дата прохождения теста. Используется для расчёта 60-дневного кулдауна.
<code>last_bot_interaction</code>	timestampz	Когда пользователь последний раз что-то писал/нажимал в боте. Используется в админке и в напоминаниях.
<code>reminded_at</code>	timestampz	Когда отправлено сообщение «Доступ открыт» через 60 дней. Чтобы не слать дважды.
<code>bot_quiz_step</code>	int4	На каком шаге анкеты-через-24-часа пользователь. Значения: 0 — не начата, 1/2/3 — отвечает на вопросы, 4 — финальное предложение, 5 — финал нажат. Защищает от повторных нажатий.
<code>referred_by</code>	bigint	Telegram ID пригласившего (берётся из реферальной ссылки <code>start=ref_<tgId></code>). NULL — пришёл сам.

Колонка	Тип	Назначение (комментарий)
<code>referrer_id</code>	UUID	UUID профиля пригласившего (вычисляется из <code>referred_by</code> через триггер <code>sync_referred_by_to_referrer</code>).
<code>referral_confirmed</code>	bool	true, когда новый пользователь прошёл тест и его реферал был засчитан пригласившему. Защита от двойного начисления.
<code>referral_confirmed_at</code>	timestampz	Когда именно реферал был подтверждён.
<code>invites_count</code>	int4	Сколько друзей этого пользователя прошли тест по его ссылке. При значении 2 запускается выдача второй опоры.
<code>shared_at</code>	timestampz	Когда пользователь нажал «Поделиться» на странице «Реферальная ссылка». От этого момента отсчитывается таймер 12 часов для сообщения «Условие не выполнено». NULL — ещё не нажимал.
<code>contact_author_clicked</code>	bool	true — пользователь нажал кнопку «Написать Еве» (где-либо). Используется в админке для аналитики.
<code>created_at</code>	timestampz	Когда профиль создан.

Таблица `test_results`

Один пользователь = одна запись (UPSERТ по `tg_id`). При повторном прохождении теста через 60+ дней запись **перезаписывается**.

Колонка	Тип	Назначение
<code>id</code>	UUID, PK	Внутренний id записи.
<code>tg_id</code>	bigint, UNIQUE	Telegram ID пользователя.
<code>profile_id</code>	UUID, FK → profiles	Ссылка на профиль.
<code>primary_support</code>	text	Буква доминирующей опоры: <code>S/U/P/R/K</code> .
<code>secondary_support</code>	text	Буква второй опоры по баллам (для смешанных текстов).
<code>score_s</code>	int	Баллы по шкале «Героическая» (Самоценность).
<code>score_u</code>	int	Баллы по шкале «Подстраивающаяся» (Угодничество).
<code>score_p</code>	int	Баллы по шкале «Перфекционирующая» (Перфекционизм).
<code>score_r</code>	int	Баллы по шкале «Удерживающая» (Контроль/эмпатия).
<code>score_k</code>	int	Баллы по шкале «Контролирующая» (Сверхбдительность).
<code>answers</code>	jsonb	Массив ответов: <code>[{questionId, score}, ...]</code> .
<code>created_at</code>	timestampz	Когда тест пройден.

Таблица `qualifications`

Ответы на 3 вопроса анкеты-через-24-часа в чат-боте. Один пользователь = одна запись (UPSERT по `profile_id`).

Колонка	Тип	Назначение
<code>id</code>	UUID, PK	Внутренний id.
<code>profile_id</code>	UUID, UNIQUE, FK → profiles	На какой профиль относится анкета.
<code>current_tension_sphere</code>	text	Ответ на вопрос 1 (сфера): <code>money</code> / <code>relations</code> / <code>health</code> / <code>other</code> / <code>everywhere</code> .
<code>tension_severity</code>	text	Ответ на вопрос 2 (острота): <code>hard</code> / <code>medium</code> / <code>light</code> .
<code>previous_experience</code>	text	Ответ на вопрос 3 (опыт): <code>yes</code> / <code>some</code> / <code>no</code> .
<code>created_at</code>	timestampz	Когда анкета заполнена.

Таблица `bot_tasks_queue`

Очередь отложенных задач, которые надо выполнить в боте в определённое время. Каждую минуту Edge Function `process-bot-queue` вычитывает `status = 'pending'` и `run_at <= NOW()`, выполняет, ставит `status = 'processed'`.

Колонка	Тип	Назначение
<code>id</code>	UUID, PK	Внутренний id задачи.
<code>profile_id</code>	UUID, FK → profiles	Кому относится задача.
<code>tg_id</code>	bigint	Telegram ID получателя сообщения.
<code>event_type</code>	text	Тип события: см. ниже.
<code>run_at</code>	timestampz	Когда выполнить задачу.
<code>status</code>	text	<code>pending</code> / <code>processing</code> / <code>processed</code> / <code>failed</code> .
<code>error_message</code>	text	Если <code>failed</code> — текст ошибки.
<code>created_at</code>	timestampz	Когда задача поставлена.
<code>updated_at</code>	timestampz	Когда последний раз обновлена.

Типы задач (`event_type`):

Тип	Что делает	Когда ставится
<code>start_qualification</code>	Шлёт связку из 3 сообщений анкеты (см. сообщение №8) и сразу первый вопрос.	Через 24 часа после первого прохождения теста.
<code>send_gift</code>	Шлёт видео-подарок (когда будет <code>file_id</code>).	Через 1 минуту после ответа на 3-й вопрос анкеты.
<code>check_referral_12h</code>	Проверяет: 2+ друга прошли тест → шлёт смешанную опору. Иначе — «Вижу, твои друзья ещё не прошли тест...».	Через 12 часов после нажатия «Поделиться».
<code>cooldown_reminder</code>	Шлёт «Хорошая новость! Доступ к тесту снова открыт...».	Через 60 дней после прохождения теста.

Важно: задача `cooldown_reminder` ставится в очередь сразу при сохранении результата теста. Альтернатива (старая) — Edge Function `send-periodic-reminders`, которая раз в сутки сканирует профили — оставляем как fallback, но основным каналом делаем очередь.

RPC-функция `submit_test_result_v2`

Атомарная серверная функция. Вызывается из `/api/test/submit`.

Делает в одной транзакции:

1. INSERT/UPDATE в `test_results` (UPSERT по `tg_id`).
2. UPDATE в `profiles`: `current_step = NULL`, `question_order = NULL`, `reminded_at = NULL`, `last_test_date = NOW()`.
3. Если у пользователя есть `referred_by` и `referral_confirmed = false` — увеличивает `invites_count` пригласившего на 1 и ставит `referral_confirmed = true`.

Возвращает `{referral_processed: bool, inviter_id: uuid | null}`.

2. Как приложение узнаёт, что пользователь подписан

Главный принцип: проверяем подписку каждый раз при открытии webapp. Кешированное значение `profiles.is_subscribed` — только для аналитики и быстрых запросов в админке.

Поток:

1. Пользователь открывает webapp (через кнопку в боте или меню «Тест»).

2. На клиенте срабатывает `Gatekeeper`, который вызывает `GET /api/user/status?tg_id=<...>`.
3. Сервер делает прямой запрос `getChatMember(channelId, tgId)` к Telegram API.
4. Если статус `member` / `administrator` / `creator` — обновляет `profiles.is_subscribed = true` и возвращает `isSubscribed: true`.
5. Если `left` / `kicked` / `restricted` — обновляет `profiles.is_subscribed = false` и возвращает `isSubscribed: false`.
6. Дополнительно сервер возвращает `lastTestDate` для расчёта кулдауна.

На клиенте `Gatekeeper` решает, что показать:

если <code>!isSubscribed</code>	→ страница «Доступ закрыт»	(раздел 3.0)
иначе если <code>daysLeft > 0</code>	→ страница «Кулдаун»	(раздел 3.7)
иначе	→ страница «Стартовая»	(раздел 3.1)

где `daysLeft = ceil((COOLDOWN_MS - (now - lastTestDate)) / DAY_MS)`, `COOLDOWN_MS = 60 * 24 * 60 * 60 * 1000`.

3. Страницы Web App (всего 8)

Дизайн всех страниц — единый, по приложенному HTML. Цвета (`#0f141a` фон, `#8BA88E` акцент, glass-card), шрифты Newsreader (заголовки) + Manrope (текст), хедер EvaTest со спра-иконкой — **на всех страницах** (декоративный, идентичный).

3.0. Страница «Доступ закрыт»

Когда открывается: пользователь не подписан на канал (после проверки в `Gatekeeper`).

Содержимое:

- Хедер EvaTest.
- Иконка закрытого замка в glass-card круте (см. дизайн).
- Заголовок: «Доступ ограничен».
- Текст: «Подпишись на мой канал. Там я даю эксклюзивные практики и знания, которых нет в открытом доступе.»
- Кнопка  Подписаться — открывает `https://t.me/sprosievu`. Свернёт webapp; вернувшись назад, пользователь снова попадёт на эту же страницу.
- Кнопка  Я подписалась — вызывает `POST /api/subscription/confirm` с `tgId`. Сервер делает `getChatMember` к Telegram. Если подписка подтвердилась:
 1. `profiles.is_subscribed = true`.

2. **Только при первом подтверждении** (`(wasAlreadySubscribed === false)`) шлём в чат-бот сообщение №2 «Подписка подтверждена!» (см. `TEXTS.md`).
3. Перезагружаем `Gatekeeper` → пользователь автоматически попадает на «Стартовую».

Если не подтвердилось — показываем красный текст: «Подписка не найдена. Пожалуйста, подпишитесь на канал и попробуйте снова.»

- Подпись внизу мелким шрифтом: «Как только подпишешься — откроется доступ ко всему функционалу теста».
-

3.1. Страница «Стартовая»

Когда открывается: пользователь подписан, и кулдаун (60 дней с последнего теста) не активен.

Содержимое (по дизайну):

- Хедер EvaTest.
- Декоративный блок с цитатой/нарративом.
- Главный текст: «У каждого человека есть внутренняя «опора» — способ держать себя в жизни. Часто эта опора искажается. Снаружи это выглядит как характер или привычки, а на деле — устойчивый механизм, который:» далее три пункта (повторяет сценарии / создаёт проблемы / не даёт выйти из круга), и финал: «Сейчас ты увидишь, какая опора у тебя доминирует».
- Две плашки-стат: `25 вопросов` и `5-7 минут`.
- Блок «Инструкция»: «Отвечай «Да», если откликается, узнаёшь себя или есть телесная реакция. «Нет» — если это не про тебя. Не думай слишком долго, доверяй ощущениям.»
- Кнопка `Начать тест` (внизу страницы) — переход на `/test`.

Что делает страница в фоне: ничего, страница чисто декларативная. Прогресс начинается на `/test`.

3.2. Страница «Тест»

Когда открывается: пользователь нажал «Начать тест» на стартовой.

Содержимое:

- Хедер EvaTest со спра-иконкой.
- **Слева сверху** кнопка «Назад» в виде стрелочки `←` — возвращает на предыдущий вопрос (на 1-м вопросе скрыта).
- **Справа сверху** счётчик `3/25` (текущий/всего).

- Под хедером — тонкая линия прогресса: $(\text{текущий} + 1) / 25 * 100\%$.
- По центру — текст текущего вопроса.
- Внизу — две большие кнопки **Да** / **Нет**.

Логика поведения:

1. При открытии:

- `GET /api/test/progress?tgId=<...>`.
- Если в ответе `isCooldown: true` → редирект на «Кулдаун».
- Если `question_order === null` → сервер сам генерирует случайный порядок и сохраняет.
- Если `current_step > 0` → показываем сразу с этого вопроса (восстанавливаем прогресс).
- Иначе — с 1-го.
- Если есть `answers` (после возврата) — показываем визуальное состояние «уже выбрано».

2. Нажатие «Да» (1) или «Нет» (0):

- Локально записываем в `answersMap[questionId] = score`.
- Через 200мс переходим к следующему вопросу.
- В фоне через `keepalive: true` шлём `PATCH /api/test/progress` с `step = currentIndex + 1`, чтобы сохранить прогресс на случай выхода из приложения.

3. Нажатие «Назад»:

- Уменьшаем `currentIndex`, чистим `selected`.

4. После 25-го ответа:

- На экране показываем оверлей: «Сейчас посчитаю. Это не совсем то, что ты думаешь.» (+ спиннер). **Минимум 2.5 секунды**, даже если сервер ответил быстрее — для UX.
- Параллельно шлём `POST /api/test/submit` с `{answers, tgId}` и `Authorization: Bearer <jwt>`.
- Сервер вызывает RPC `submit_test_result_v2`, ставит задачу `start_qualification` в очередь на +24 часа и задачу `cooldown_reminder` на +60 дней.
- При успехе сервер возвращает `{dominantTrait, referralProcessed}`. Сохраняем в `sessionStorage.eva_result`.
- **Параллельно** Supabase webhook от вставки в `test_results` триггерит Edge Function `process-bot-notifications`, которая шлёт пользователю в чат-бот сообщение №3 «Твоя опора» (см. п. 4).

- Редирект на `/result`.

Удалить из текущего кода:

- Текст «Осмысляю...» в нижнем спиннере (оставить только «Сейчас посчитаю. Это не совсем то, что ты думаешь.»).

3.3. Страница «Результат»

Когда открывается: пользователь только что прошёл тест, или открывает webapp после прохождения и попадает сюда напрямую (если есть `test_results`).

Содержимое (фаза 1 — сразу после теста):

- Хедер EvaTest.
- **Hero-секция:** картинка соответствующей опоры (см. таблицу маппинга ниже) + плашка «РЕЗУЛЬТАТ ТЕСТА» + заголовок «Ваша доминирующая опора: [Название]».
- **Glass-card с описанием:** полный длинный текст из `lib/constants/results.ts → FULL_RESULTS_TEXTS[trait]` (со всеми блоками «Цена:», «⚡ Внутри звучит:» и т. д.).
- **Блок «Удивил ли тебя результат?»** с двумя кнопками: `Да` / `Нет`.

Маппинг буква → название → картинка:

Буква	Название	Файл картинки
S	Героическая опора	<code>/hero.png</code>
U	Подстраивающаяся опора	<code>/pleaser.png</code>
P	Перфекционирующая опора	<code>/perfectionist.png</code>
R	Удерживающая опора	<code>/stayer.png</code>
K	Контролирующая опора	<code>/controller.png</code>

Действие при нажатии `Да` или `Нет`:

Никаких модальных-подтверждений! (старая модалка «Уверена? Ты не сможешь изменить свой ответ» полностью удалена.)

- Сразу появляется задержка 1 секунда (с маленьким спиннером).
- После задержки на той же странице снизу появляется **новый блок** (с вертикальной линией слева, как в дизайне) и страница автоматически **скроллится** к нему.

Текст блока зависит от выбора:

- После ☐ Да:
 - «Это нормально. Более того — это и есть часть механизма. Искажённая опора устроена так, что ты не видишь её напрямую. И именно поэтому снова и снова оказываешься в одних и тех же ситуациях.»
 - «Базовая опора — это только часть картины. Есть ещё смешанные конфигурации, которые активируются в стрессе.»
- После ☐ Нет:
 - «Это говорит о том, что ты уже явно не новичок в самопознании. Ты видишь этот паттерн, но это ещё не меняет ситуацию.»
 - «Базовая опора — это только часть картины. Есть ещё смешанные конфигурации, которые активируются в стрессе.»

Под этим блоком сразу появляется следующий блок (фаза 3):

«Хочешь увидеть свою вторую искажённую опору?»

Две кнопки:

- ☐ Да — переход на страницу «Доступ» (раздел 3.4).
- ☐ Не уверена — переход на страницу «Механизм» (раздел 3.6).

Обе кнопки можно нажать только один раз (после нажатия страница меняется).

3.4. Страница «Доступ»

Когда открывается: с «Результата» по кнопке ☐ Да под вопросом про вторую опору.

Содержимое:

- Хедер EvaTest.
 - Декоративная картинка/иллюстрация (по дизайну).
 - Цитата: «Я обычно открываю этот слой только тем, кто идёт в работу. Потому что важно не просто увидеть, а понять, как это устроено и что с этим делать.»
 - Текст ниже: «Сейчас ты можешь получить разбор своей второй опоры, если пригласишь 2 друзей в бота и они пройдут тест. Я даю этот доступ в обмен на расширение проекта.»
 - Одна кнопка ☐ Получить ссылку — переход на страницу «Реферальная ссылка» (раздел 3.5).
-

3.5. Страница «Реферальная ссылка»

Когда открывается: с «Доступ» по кнопке «Получить ссылку».

Содержимое:

- Хедер EvaTest.
- Декоративная картинка/символ (по дизайну).
- Подзаголовок: «Вот твоя персональная ссылка:».
- Поле со ссылкой `(https://t.me/sprosievubot?start=ref_<tgId>)` + кнопка-иконка с символом копирования (при нажатии копирует в буфер обмена, на 1.5 секунды показывает галочку).
- Подпись: «Когда 2 человека перейдут по ней и пройдут тест, я вышлю тебе твою вторую искажённую опору.»
- Кнопка `(Поделиться)` — основное действие.

Действие кнопки «Поделиться»:

1. Шлём `(POST /api/share)` с `({tgId, link})`.
2. Сервер проверяет `(profiles.shared_at)`:
 - Если `(NULL)` (первый раз): а. Шлёт в чат-бот **два сообщения подряд** (тексты — см. `(TEXTS.md)` №4). б. Ставит `(profiles.shared_at = NOW())`. в. Создает в `(bot_tasks_queue)` задачу `(check_referral_12h)` с `(run_at = NOW() + 12 часов)`.
 - Если уже не `(NULL)` — ничего не делает (повторное нажатие не должно ничего повторно слать).
3. Клиент получает `(success: true)`, ставит `(localStorage.eva_shared_at)`, **закрывает webapp** через `(WebApp.close())`.

Важно: ссылку пользователь может копировать и пересылать сколько угодно. Сервер просто отслеживает, выполнено ли условие реферальной системы (см. п. 5).

3.6. Страница «Механизм»

Когда открывается: с «Результата» по кнопке `(Не уверена)` под вопросом про вторую опору.

Содержимое:

- Хедер EvaTest.
- Декоративная анимация (пульсирующий круг, см. дизайн).
- Цитата: «Ты сейчас увидела механизм. И, скорее всего, это не первый раз, когда ты что-то про себя понимаешь. Вопрос в другом: почему это до сих пор не меняет твою жизнь? Потому что понимание не демонтирует паттерн. Это делается только через работу.»
- Текст: «Сейчас нет смысла проходить тест повторно. У тебя есть 2 месяца, чтобы демонтировать доминирующую опору. Через 2 месяца ты сможешь снова пройти тест и

увидеть динамику.»

- Две кнопки:
 - `Узнать вторую опору сейчас` — переход на страницу «Доступ» (раздел 3.4).
 - `Узнаю через 2 месяца` — `Telegram.WebApp.close()` (просто закрывает webapp, ничего в бот не шлёт).
-

3.7. Страница «Кулдаун»

Когда открывается: пользователь подписан, но открыл webapp в течение 60 дней после прохождения теста.

Содержимое (по дизайну):

- Хедер EvaTest.
- Декоративная анимация (песочные часы / пульс).
- Заголовок: «Время для действий».
- Цитата: «Сейчас нет смысла проходить тест повторно. У тебя есть время, чтобы демонтировать доминирующую опору. Позже ты сможешь снова пройти тест и увидеть динамику.»
- Glass-card с прогрессом: «Тест будет доступен снова через N дней» (где N — динамическое значение).
- Одна кнопка `Написать Еве` — открывает диалог с `@evapatrakhina` через `WebApp.openTelegramLink('https://t.me/evapatrakhina')`. Перед открытием шлёт `POST /api/user/contact-author` с `tgId` (ставит `profiles.contact_author_clicked = true`).

Кнопки «Пройти тест» здесь нет — тест физически недоступен.

3.8. Страница «Административная панель»

Когда открывается: на любой странице 5 кликов в верхней части экрана (`y < 50px`) в течение 2 секунд → промпт с PIN-кодом → правильный код → редирект на `/admin`.

PIN-код: `0999` (установить через переменную окружения `ADMIN_PIN`); в коде `app/api/admin/login/route.ts` дефолт меняем с `2026` на `0999`).

Функционал страницы: оставляем как есть (статистика пользователей, рассылка, управление ссылками-подарками).

Дизайн страницы: обновить под общий — те же шрифты Newsreader/Manrope, цвета `#0f141a` / `#8BA88E`, glass-card блоки, хедер EvaTest. Но функционал не трогаем.

4. Сообщения чат-бота (всего 9)

Все точные тексты — в `TEXTS.md`. Здесь — только триггеры и условия.

Сообщение №1 — «Старт»

- **Триггер:** пользователь отправил `/start` боту в первый раз (профиля ещё нет).
- **Содержимое:** картинка `start.png` + текст из `TEXTS.md` №1 + две inline-кнопки: `Подписаться` (URL → канал) и  `Я подписалась` (callback `check_sub`).
- **Что делает callback `check_sub`:** проверяет подписку через `getChatMember`. Если подтвердилась → ставит `is_subscribed = true`, шлёт сообщение №2.

Сообщение №2 — «Подписка подтверждена»

- **Триггер:**
 - Callback `check_sub` в боте (когда пользователь нажал «Я подписалась» в боте).
 - **ИЛИ** успешный `POST /api/subscription/confirm` из webapp при первом подтверждении (`wasAlreadySubscribed === false`).
- **Содержимое:** картинка `start1.png` + текст из `TEXTS.md` №2 + кнопка  `Пройти тест` (web_app, открывает webapp).
- **Защита от дублей:** если пользователь уже был `is_subscribed = true`, сообщение НЕ шлём повторно.

Сообщение №3 — «Твоя опора»

- **Триггер:** Supabase database webhook на INSERT/UPDATE в `test_results` → Edge Function `process-bot-notifications`.
- **Содержимое:** картинка опоры (см. таблицу п. 3.3) + полный текст опоры из `TEXTS.md` №3.
- **Защита от дублей:** при UPDATE проверяем, что `primary_support` действительно изменился (иначе пропускаем).

Сообщение №4 — «Реферальная ссылка» (2 сообщения подряд)

- **Триггер:** первое нажатие кнопки «Поделиться» на странице «Реферальная ссылка».
- **Содержимое:**
 - Сообщение 4а: текст из `TEXTS.md` №4а («Скопируй сообщение ниже и отправь его друзьям...»).
 - Сообщение 4б: текст из `TEXTS.md` №4б + реферальная ссылка `https://t.me/sprosievubot?start=ref_<tgId>`.

- **Защита от дублей:** только первое нажатие (`shared_at IS NULL`).

Сообщение №5 — «Условие не выполнено»

- **Триггер:** Edge Function (`process-bot-queue`) обрабатывает задачу (`check_referral_12h`) через 12 часов после (`shared_at`). Если (`invites_count < 2`), шлёт это сообщение.
- **Содержимое:** текст из (`TEXTS.md`) №5. Без кнопок.

Сообщение №6 — «Вторая опора» (смешанная)

- **Триггер:** (`invites_count`) достиг 2.
 - Может произойти **двумя путями**:
 - В пределах 12 часов — обработчик задачи (`check_referral_12h`) сначала проверит условие и вместо «Условие не выполнено» отправит это сообщение.
 - После 12 часов — в (`webhook/telegram`) или RPC (`submit_test_result_v2`), когда новый реферал прошёл тест, прибавил (`invites_count`) до 2 и условие первый раз выполнилось.
- **Содержимое:** текст смешанной опоры из (`lib/telegram.ts` → `MIXED_TRAIT_TEXTS[<key>]`), где (`<key>`) — отсортированный код двух опор (например, для S+R это (`RS`)).
- **Защита от дублей:** одна попытка отправки. Если уже отправлялось — флаг (например, (`mixed_trait_sent = true`) в `profiles`, добавить миграцию).

Сообщение №7 — «Доступ открыт» (через 60 дней)

- **Триггер:** Edge Function (`process-bot-queue`) обрабатывает задачу (`cooldown_reminder`) через 60 дней после (`last_test_date`). И (`reminded_at IS NULL`) (защита от повторов).
- **Содержимое:** только текст из (`TEXTS.md`) №7 + кнопка ( Пройти тест) (`web_app`).
- **После отправки** — ставим (`reminded_at = NOW()`).

Сообщение №8 — «Анкета» (через 24 часа)

└ Самый сложный блок. Цепочка из вступительных сообщений и интерактивного опроса.

Триггер: Edge Function (`process-bot-queue`) обрабатывает задачу (`start_qualification`) через 24 часа после первого прохождения теста (RPC (`submit_test_result_v2`) ставит её в очередь).

Защита от дублей: перед отправкой проверяем (`profiles.bot_quiz_step`). Если (`>= 1`) — анкета уже была начата, ничего не делаем.

Шаги цепочки (всё без задержек, одно за другим):

1. Сообщение 8а: текст из `TEXTS.md` №8а («Ты уже заметила, как искажённая опора...»).
2. Сообщение 8б: текст из `TEXTS.md` №8б («Ты увидела механизм. И, скорее всего...»).
3. Вопрос 1: текст «В какой сфере сильнее чувствуется напряжение?» + 5 callback-кнопок: `Деньги`, `Отношения`, `Здоровье`, `Другое`, `Везде` (`callback_data: quiz_q1_money` и т.п.).
4. После ответа на Q1 — `bot_quiz_step = 2`, сохраняем `current_tension_sphere`, шлём вопрос 2: «Насколько остро это ощущается?» + 3 кнопки: `Сильно мешает`, `Пока терпимо`, `Фоновое`.
5. После ответа на Q2 — `bot_quiz_step = 3`, сохраняем `tension_severity`, шлём вопрос 3: «Пробовала ли что-то делать?» + 3 кнопки: `Да, много`, `Немного`, `Нет`.
6. После ответа на Q3:
 - `bot_quiz_step = 4`, сохраняем `previous_experience`.
 - **Сразу** шлём финальное сообщение из `TEXTS.md` №8в («Есть 2 способа работы...») + 3 кнопки: `Жесткий быстрый` (`quiz_final_hard`), `Мягкий постепенный` (`quiz_final_soft`), `Пока не готова` (`quiz_final_not_ready`).
 - **Параллельно** ставим в `bot_tasks_queue` задачу `send_gift` с `run_at = NOW() + 60 секунд`.
7. Через 1 минуту обработчик отправляет видео-подарок (`sendVideo` с `protect_content: true`), используя `GIFT_VIDEO_FILE_ID` из env). **Пока `file_id` не получен — отправляется текстовая заглушка** «📺 Твой подарок готов!».

Кнопки финального сообщения:

Кнопка	callback	Действие
<code>Жесткий быстрый</code>	<code>quiz_final_hard</code>	Шлём в бот сообщение «Отлично! Нажми кнопку ниже...» с inline-кнопкой URL <code>https://t.me/evapatrakhina?text=Пробой!</code> . Ставим <code>bot_quiz_step = 5</code> , <code>contact_author_clicked = true</code> .
<code>Мягкий постепенный</code>	<code>quiz_final_soft</code>	То же, но с текстом <code>Пирамида Потенциала</code> .
<code>Пока не готова</code>	<code>quiz_final_not_ready</code>	Просто ставим <code>bot_quiz_step = 5</code> .

Защита от повторов: при `bot_quiz_step = 5` callback-кнопки больше не реагируют. После любого нажатия inline-keyboard убирается через `editMessageReplyMarkup` (как сейчас в коде).

Важное замечание: в текущем коде после нажатия одной из 3 финальных кнопок отправлялось сообщение «Благодарю тебя за честность...». В `logic.md` (Г4) ты явно указал: **под финальным сообщением ничего не присылается, потому что практика — это видео-подарок, который придёт через минуту.** Поэтому это сообщение удаляем.

Сообщение №9 — «Любое сообщение»

- **Триггер:** пользователь отправляет в бот любой текст, который не является командой (`/start`, `/test`).
- **Содержимое:** «Рада видеть тебя снова, `[first_name]!`» (если `first_name` есть, иначе просто «Рада видеть тебя снова!») + кнопка URL `Написать Еве` → `https://t.me/evapatrakhina`.

Старая логика «Открыть приложение» в ответ на любой текст — удаляется.

5. Реферальная система — полный сценарий

Шаги жизни реферальной ссылки:

1. Пользователь А прошёл тест, нажал «Поделиться» на странице «Реферальная ссылка».
2. Сервер шлёт в чат-бот А 2 сообщения с инструкцией и ссылкой `https://t.me/sprosievubot?start=ref_<tgIdA>`.
3. `profiles.shared_at = NOW()` для А.
4. В `bot_tasks_queue` добавляется задача `check_referral_12h` с `run_at = NOW() + 12h`, `tg_id = tgIdA`.
5. Webapp у А закрывается. Ссылка остаётся у А в чат-боте — он может ей делиться сколько угодно и когда угодно.
6. Пользователь В перешёл по ссылке → бот делает `upsert` в `profiles` с `tg_id = tgIdB` и `referred_by = tgIdA`.
7. В подписывается на канал и проходит тест.
8. RPC `submit_test_result_v2` при сохранении результата В видит `referred_by = tgIdA`, `referral_confirmed = false` → ставит `referral_confirmed = true` и +1 к `invites_count` пригласившему (А).
9. Тот же сценарий для пользователя С → у А `invites_count = 2`.
10. Edge Function `process-bot-notifications` слушает UPDATE `profiles`. Видит, что `invites_count` стал 2 и раньше был < 2 → шлёт А сообщение №6 «Вторая опора».
11. **Если 12 часов уже прошли** — это путь №2 для сообщения №6, всё работает само.
12. **Если 12 часов ещё не прошли** — обработчик задачи `check_referral_12h` дополнительно проверит флаг (например, новый `mixed_trait_sent`) и не отправит

повторно.

Если через 12 часов `invites_count < 2`:

- Обработчик задачи `check_referral_12h` шлёт сообщение №5 «Условие не выполнено».
- На этом задача `check_referral_12h` помечается `processed`. Больше не запускается.
- Но логика п. 10 продолжает работать вечно: как только когда-нибудь А наберёт 2 реферала, он получит сообщение №6.

Условия флага дублирования:

Чтобы сообщение №6 не отправилось дважды (один раз обработчиком 12-часов, другой раз watchdog'ом UPDATE) — добавляем в БД флаг `profiles.mixed_trait_sent BOOLEAN DEFAULT FALSE` (новая миграция). Перед отправкой проверяем — если уже `true`, не шлём.

6. Полный поток жизни пользователя

1. `/start` в боте → создаётся profile → сообщение №1 «Старт»
2. Подписка → callback `check_sub` или `/api/subscription/confirm`
→ сообщение №2 «Подписка подтверждена»
3. Открытие webapp → Gatekeeper:
 - не подписан → стр. «Доступ закрыт» (3.0)
 - подписан + кулдаун → стр. «Кулдаун» (3.7)
 - подписан + нет кулдауна → стр. «Стартовая» (3.1)
4. «Начать тест» → стр. «Тест» (3.2) → 25 вопросов
5. После 25-го → оверлей 2.5 сек «Сейчас посчитаем...»
→ RPC `submit_test_result_v2`
→ ставим задачу `start_qualification` (+24ч)
→ ставим задачу `cooldown_reminder` (+60д)
→ webhook → сообщение №3 «Твоя опора» в бот
→ стр. «Результат» (3.3)
6. «Удивил ли тебя?» → Да/Нет, инсайт-блок
7. «Хочешь увидеть вторую?» → Да → стр. «Доступ» (3.4)
→ Не уверена → стр. «Механизм» (3.6)
- 8a. Из «Доступ» → «Получить ссылку» → стр. «Реферальная» (3.5)
→ «Поделиться» → сообщения №4 в бот
→ ставим задачу `check_referral_12h` (+12ч)
→ `webapp.close()`
- 8b. Из «Механизм» → «Узнать вторую опору сейчас» → стр. «Доступ» (3.4)
→ «Узнаю через 2 месяца» → `webapp.close()`
9. Через 24 ч → `start_qualification` → сообщения №8 + анкета
10. После Q3 → +1 минута → сообщение «🎁 подарок» (видео)
11. Через 12 ч → `check_referral_12h`:
 - 2+ → сообщение №6 «Вторая опора»
 - <2 → сообщение №5 «Условие не выполнено»

12. Когда-нибудь набирает 2 → сообщение №6 (если ещё не отправлено)
13. Через 60 дней → cooldown_reminder → сообщение №7 «Доступ открыт»
→ reminded_at = NOW
14. Снова /test → доступ к тесту разблокирован, цикл повторяется

7. Что нужно изменить в существующем коде

7.1. Файлы webapp (frontend)

Файл	Что делаем
app/page.tsx	Полный редизайн под HTML. Удалить ветку с кулдауном (вынесли в отдельную страницу). Кнопка <code>Начать тест</code> (не «Пройти тест»). Хедер <code>EvaTest</code> .
app/test/page.tsx	Редизайн. Удалить «Осмысляю...». Кнопка «Назад» — стрелочка слева сверху. Хедер <code>EvaTest</code> .
app/result/page.tsx	Редизайн. Удалить <code>ConfirmModal</code> . Удалить экран <code>cooldown-message</code> из <code>funnelStep</code> . Добавить переход «Не уверена» → <code>/mechanism</code> . Исправить читаемые поля: <code>API</code> <code>/api/test/results</code> возвращает <code>primarySupport/secondarySupport</code> , а клиент читал <code>dominantTrait/secondaryTrait</code> — синхронизировать.
app/mechanism/page.tsx	Создать новую страницу.
app/cooldown/page.tsx	Создать новую страницу. Чтобы не дублировать кулдаун в <code>app/page.tsx</code> и <code>Gatekeeper.tsx</code> .
app/admin/page.tsx	Только редизайн под общий стиль. Функционал не трогаем.

Файл	Что делаем
<code>components/Gatekeeper.tsx</code>	Редизайн «Доступ закрыт» под HTML. Логика: <code>not_subscribed</code> → текущий экран; <code>cooldown</code> → редирект на <code>/cooldown</code> ; <code>not_subscribed</code> определять каждый раз заново через API.
<code>components/AdminEntrance.tsx</code>	Без изменений.
<code>components/results/ResultScreen.tsx</code>	Редизайн под HTML.
<code>components/results/SurpriseResponseScreen.tsx</code>	Редизайн. Тексты — из <code>TEXTS.md</code> . Кнопки <code>Да</code> / <code>Не уверена</code> .
<code>components/results/ReferralGateScreen.tsx</code>	Редизайн. Это страница «Доступ» (3.4). Текст и кнопка по <code>TEXTS.md</code> . Только одна кнопка <code>Получить ссылку</code> .
<code>components/results/ReferralLinkScreen.tsx</code>	Редизайн. Подзаголовок «Вот твоя персональная ссылка:». Текст по <code>TEXTS.md</code> .
<code>components/results/CooldownScreen.tsx</code>	Удаляем — кулдаун теперь отдельная страница.
<code>components/ConfirmModal.tsx</code>	Удаляем — модалка не используется.

7.2. API роуты (backend)

Файл	Что делаем
<code>app/api/auth/route.ts</code>	Без изменений.
<code>app/api/user/status/route.ts</code>	Изменить: теперь не доверяет кешу <code>is_subscribed</code> . При каждом запросе делает <code>getChatMember</code> к Telegram, обновляет БД, возвращает свежее значение.
<code>app/api/subscription/confirm/route.ts</code>	Поправить URL картинки: вместо хардкода <code>https://eva-9udm.vercel.app/start1.png</code> использовать <code>process.env.NEXT_PUBLIC_APP_URL</code> . Текст сообщения — из <code>TEXTS.md</code> №2 (полный).
<code>app/api/test/progress/route.ts</code>	Без изменений.

Файл	Что делаем
app/api/test/submit/route.ts	Добавить вставку задачи <code>cooldown_reminder</code> в <code>bot_tasks_queue</code> с <code>run_at = NOW() + 60 days</code> .
app/api/test/results/route.ts	Исправить контракт ответа: возвращать <code>dominantTrait</code> (не <code>primarySupport</code>) и <code>secondaryTrait</code> — чтобы совпадало с тем, что ожидает клиент <code>app/result/page.tsx</code> .
app/api/qualification/submit/route.ts	Удаляем целиком — анкета теперь полностью в боте, не в webapp.
app/api/share/route.ts	Текст 2 сообщений — из <code>TEXTS.md</code> №4а и №4б. Добавить вставку задачи <code>check_referral_12h</code> в очередь.
app/api/user/share-clicked/route.ts	Удаляем целиком — дубликат <code>/api/share</code> .
app/api/user/contact-author/route.ts	Без изменений.
app/api/notify-author/route.ts	Проверить, используется ли. Если нет — удалить.
app/api/admin/login/route.ts	Поменять дефолтный PIN с <code>2026</code> на <code>0999</code> .
app/api/admin/*	Функционал не трогаем.
app/api/webhook/telegram/route.ts	Большая чистка (см. ниже).
app/api/bot/setup/route.ts	Без изменений (только название кнопки в меню оставляем <code> Тест</code>).
app/api/bot/send-gift-message/route.ts	Удаляем — старая нерабочая логика.
app/api/gift-link/route.ts	Оставляем, используется в админке.
app/api/referrals/route.ts	Проверить актуальность.

7.3. Webhook бота — что делаем

Хэндлер	Что делаем
<code>/start <ref?></code>	Текст — из <code>TEXTS.md</code> №1. Картинка <code>start.png</code> . Кнопки <code> Подписаться</code> и <code> Я подписалась</code> .

Хэндлер	Что делаем
Callback <code>check_sub</code>	При успехе — отправляем сообщение №2 (только при первом подтверждении).
<code>/test</code>	Если подписан — открываем webapp. Если нет — показываем сообщение №1.
Текстовое сообщение (не команда)	Шлём сообщение №9 «Рада видеть тебя снова, [first_name]!» + кнопка URL <code>Написать Eve</code> .
Callback <code>quiz_q1_*</code> , <code>q2_*</code> , <code>q3_*</code>	Логика анкеты — см. п. 4 №8. Тексты вопросов — из <code>TEXTS.md</code> №8.
Callback <code>quiz_final_hard/soft/not_ready</code>	Логика — см. п. 4 №8. Удалить отправку сообщения «Благодарю за честность...».
Видео от <code>evapatrakhina</code> <code>bizbezt</code>	Дебаг-инструмент: отвечаем <code>file_id</code> . Оставляем как есть.

7.4. Edge Functions

Функция	Что делаем
<code>process-bot-queue/index.ts</code>	Полная переделка обработки <code>event_type</code> : добавить <code>start_qualification</code> , <code>check_referral_12h</code> , <code>cooldown_reminder</code> . Текст для <code>start_qualification</code> — из <code>TEXTS.md</code> №8. Видео-подарок — <code>sendVideo</code> с <code>protect_content: true</code> .
<code>process-bot-notifications/index.ts</code>	Сообщение №3 — текст из <code>TEXTS.md</code> №3 (синхронизировать с <code>lib/constants/results.ts</code>). Сообщение №6 — добавить флаг <code>mixed_trait_sent</code> для защиты от дублей.
<code>send-periodic-reminders/index.ts</code>	Оставляем как fallback. Текст — из <code>TEXTS.md</code> №7.

7.5. Библиотеки

Файл	Что делаем
<code>lib/questions.ts</code>	Без изменений.
<code>lib/scoring.ts</code>	Без изменений.
<code>lib/constants/results.ts</code>	Тексты из <code>TEXTS.md</code> №3 (5 опор).

Файл	Что делаем
<code>lib/telegram.ts</code>	Тексты смешанных опор — синхронизировать с <code>TEXTS.md</code> №6.
<code>lib/telegram-bot.ts</code>	Без существенных изменений. Можно добавить хелпер <code>sendGiftVideo</code> .
<code>lib/author-contact.ts</code>	Без изменений.
<code>lib/admin-auth.ts</code>	Без изменений.

7.6. Новые миграции БД

```
sql

-- Migration 107: add mixed_trait_sent flag
ALTER TABLE profiles ADD COLUMN IF NOT EXISTS mixed_trait_sent BOOLEAN DEFAULT FALSE
ALTER TABLE profiles ADD COLUMN IF NOT EXISTS mixed_trait_sent_at TIMESTAMPTZ DEFAULT
```

```
sql

-- Migration 108: extend RPC to enqueue cooldown_reminder
-- (вписываем в существующую submit_test_result_v2)
```

8. Структура переменных окружения (`.env`)

```
TELEGRAM_BOT_TOKEN=<токен бота>
TELEGRAM_CHANNEL_ID=<-100xxx>
TELEGRAM_CHANNEL_URL=https://t.me/sprosievu
TELEGRAM_WEBHOOK_SECRET=<любой строка>

NEXT_PUBLIC_BOT_USERNAME=sprosievubot
NEXT_PUBLIC_APP_URL=https://<домен>
NEXT_PUBLIC_TELEGRAM_CHANNEL_URL=https://t.me/sprosievu

NEXT_PUBLIC_SUPABASE_URL=<...>
SUPABASE_SERVICE_ROLE_KEY=<...>
SUPABASE_JWT_SECRET=<...>

ADMIN_PIN=0999
GIFT_VIDEO_FILE_ID=<пусто пока, добавим позже>
```


9. Чек-лист готовности

- ☐ Применены миграции 107, 108.
- ☐ PIN админки — `0999`.
- ☐ Все 8 страниц webapp перерисованы под HTML-дизайн.
- ☐ Удалена `ConfirmModal`, удалён `CooldownScreen`-компонент, удалены дубликаты API.
- ☐ Тексты во всех точках совпадают с `TEXTS.md` слово-в-слово.
- ☐ `cooldown_reminder` ставится в очередь при сабмите теста.
- ☐ `check_referral_12h` ставится при «Поделиться».
- ☐ `start_qualification` ставится при сабмите теста (+24ч).
- ☐ `send_gift` ставится при ответе на 3-й вопрос анкеты (+1мин).
- ☐ `process-bot-queue` обрабатывает все 4 типа задач.
- ☐ Подписка проверяется каждый раз свежим запросом к Telegram, флаг `is_subscribed` только кешируется.
- ☐ Сообщение №6 «Вторая опора» защищено от дублей флагом `mixed_trait_sent`.
- ☐ Видео-подарок: пока заглушка-текст; после получения `file_id` — заменить на `sendVideo(protect_content: true)`.
- ☐ Удалена отправка «Благодарю за честность...» после финальных кнопок анкеты.
- ☐ Сообщение №9 шлётся в ответ на любое не-команду в боте.

Работа с этим документом: если возникнет потребность изменить логику — сначала правим `LOGIC.md`, потом синхронизируем код. Никогда наоборот.